



NOTRE DAME UNIVERSITY

BANGLADESH

Computer Graphics Lab Project Report

Course Code: CSE-4204

Course Title: Computer Graphics Lab

Project Title: Egg Drop Saga (Game)

Submitted by:

Name: Istiak Alam

ID: 0692230005101005

Name: Tanvir Hossain Ratul

ID: 0692230005101004

Batch: CSE-20

Submission Date: June 9, 2026

Submitted to:

Humayara Binte Rashid

Lecturer, Dept. of CSE

Notre Dame University Bangladesh

Table of Contents

Abstract	1
1 Introduction	2
1.1 Project Background	2
1.2 Motivation	2
1.3 Objectives	3
1.4 Scope	3
1.5 Tools and Libraries Used	4
2 System Design	5
2.1 Project Structure Overview	5
2.2 System Overview	6
2.3 Game State Management	7
2.4 OpenGL Rendering Pipeline	8
2.5 Coordinate System Design	10
3 Implementation	11
4 Results and Output	12
5 Discussion	13
6 Conclusion	14
7 Future Work	15
8 References	16
Appendix	17

Abstract

Egg Drop Saga is a 2D arcade-style game developed using OpenGL and GLUT in C++ as part of the Computer Graphics Laboratory course. The project demonstrates the practical application of fundamental computer graphics concepts including coordinate systems, geometric modeling, transformations, animation, rendering pipelines, and user interaction. In the game, a player controls a bucket to catch eggs dropped by moving chickens while avoiding missed catches that reduce the player's lives. The project incorporates real-time animation, keyboard-based controls, collision detection, score tracking, audio integration, and multiple game states such as menu, gameplay, pause, and game-over screens. The visual elements are created using OpenGL primitive drawing techniques and coordinate-based object construction. The primary objective of the project is to apply theoretical graphics concepts in an interactive environment while gaining experience in event-driven programming and real-time rendering. The final implementation successfully demonstrates the integration of computer graphics principles with game development techniques.

Chapter 1 Introduction

1.1 Project Background

Computer Graphics is a branch of computer science that focuses on the creation, manipulation, and rendering of visual content using computational techniques. OpenGL is one of the most widely used graphics libraries for developing interactive graphical applications and games. As part of the Computer Graphics Laboratory course, students are encouraged to apply theoretical concepts such as coordinate systems, geometric modeling, transformations, animation, rendering, and event handling in a practical project.

The project **Egg Drop Saga** is a 2D arcade-style game developed using C++ and OpenGL (GLUT). The game simulates a farm environment where chickens continuously move and drop eggs from the top of the screen. The player controls a bucket positioned at the bottom of the screen and must catch the falling eggs to earn points while avoiding missed catches that reduce the player's lives. The project integrates graphical object modeling, real-time animation, collision detection, user interaction, and audio feedback into a complete gaming experience.

1.2 Motivation

The inspiration for this project came from classic arcade catching games that emphasize quick reaction and hand-eye coordination. These games are simple in design yet provide an engaging user experience through continuous movement, scoring systems, and increasing difficulty.

From an academic perspective, the project offered an opportunity to implement various computer graphics concepts learned throughout the course in a practical environment. Developing a complete game using OpenGL allowed the team to gain hands-on experience with rendering pipelines, object-oriented design, animation techniques, coordinate-based drawing, and event-driven programming while creating an enjoyable interactive application.

1.3 Objectives

The primary objectives of the Egg Drop Saga project are:

- To develop a fully functional 2D game using OpenGL and GLUT.
- To understand and implement the OpenGL rendering pipeline.
- To apply coordinate-based geometric modeling for creating game objects such as chickens, eggs, buckets, and backgrounds.
- To implement real-time animation for moving objects within the game environment.
- To develop keyboard-based user interaction for controlling game elements.
- To implement collision detection between falling eggs and the player's bucket.
- To create a score and life management system.
- To integrate sound effects and background music for enhanced gameplay.
- To gain practical experience in object-oriented game development using C++.

1.4 Scope

The scope of the project includes the design and implementation of a complete 2D arcade game using OpenGL. The project covers graphical object creation, rendering, animation, collision detection, score management, sound integration, and user interaction through keyboard controls.

The game contains multiple functional components including a main menu, gameplay screen, pause system, score tracking, life management, and game-over interface. All visual elements are generated using OpenGL primitives and coordinate-based drawing techniques rather than external game engines.

However, the project does not include advanced features such as 3D graphics, online multiplayer support, artificial intelligence opponents, physics engines, networking systems, or database integration. The focus remains on demonstrating core computer graphics principles within a 2D interactive environment.

1.5 Tools and Libraries Used

The development of Egg Drop Saga required several software tools and libraries, which are listed below:

Tool/Library	Purpose
C++	Primary programming language used for game development.
OpenGL	Graphics rendering library used for drawing and displaying visual objects.
GLUT (OpenGL Utility Toolkit)	Provides window management, event handling, keyboard input, and rendering callbacks.
SDL2_mixer	Used for audio playback including background music and sound effects.
Code::Blocks	Integrated Development Environment (IDE) used for coding and debugging.
MinGW GCC Compiler	Compiles and links the C++ source code.
Windows Operating System	Development and testing platform.
GitHub	Version control and project collaboration.

Table 1.1: Tools and Libraries

The combination of these tools enabled the development of an interactive OpenGL game while providing efficient debugging, code management, graphics rendering, and audio integration capabilities.

Chapter 2 System Design

2.1 Project Structure Overview

The Egg Drop Saga project follows a modular object-oriented architecture where each major game component is implemented as an independent class. This design improves code maintainability, readability, and scalability by separating rendering, game logic, audio management, and user interface functionalities into distinct modules.

The overall project directory structure is shown below:

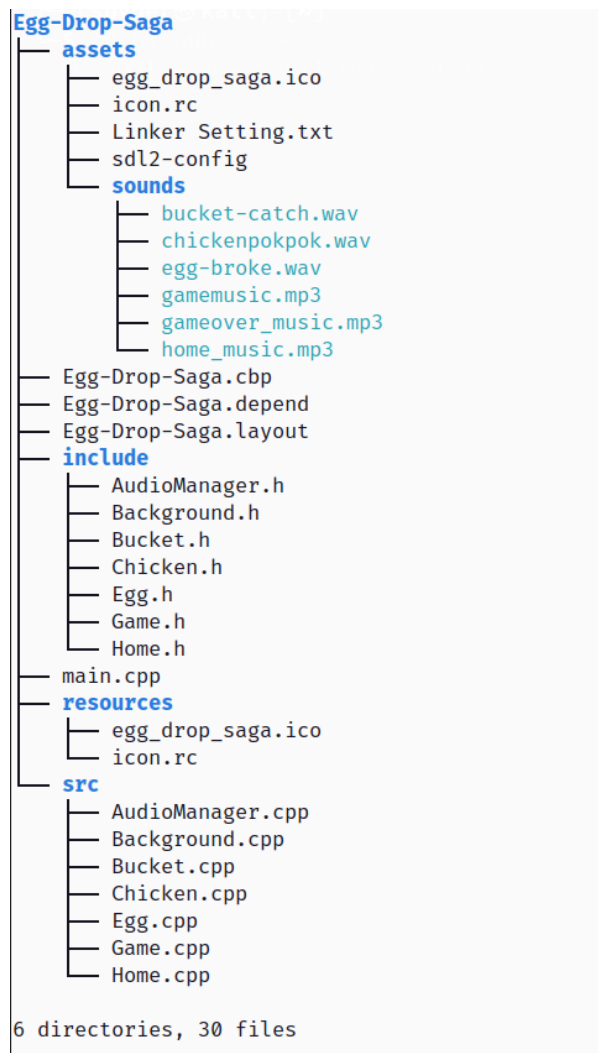


Figure 2.1: Project File Structure

The purpose of each major component is described below:

Module	Responsibility
main.cpp	Entry point of the application. Initializes OpenGL, creates the game window, registers callback functions, and starts the GLUT event loop.
Game Class	Controls the overall game state, score system, life management, collision detection, rendering coordination, and gameplay flow.
Chicken Class	Creates and animates the chickens that move horizontally and drop eggs.
Egg Class	Manages egg creation, falling animation, collision checking, and respawning logic.
Bucket Class	Represents the player-controlled bucket used to catch falling eggs.
Background Class	Draws the complete game environment including sky, ground, scenery, decorative objects, and visual layers.
Home Class	Handles the home screen, menu interface, instructions, and navigation before gameplay begins.
AudioManager Class	Controls background music, sound effects, audio loading, playback, and stopping mechanisms using SDL2_mixer.
Assets Directory	Stores sound effects, music files, application icons, and additional project resources.

Table 2.1: Project Modules and Responsibilities

The modular architecture ensures that graphical rendering, user interaction, animation, collision detection, and audio processing remain logically separated, making the project easier to understand, debug, and extend in future versions.

2.2 System Overview

Egg Drop Saga is implemented using an event-driven architecture based on OpenGL and GLUT. The system is organized around a central Game class that manages gameplay logic, rendering operations, collision detection, animation updates, score management, and state transitions.

The application begins execution from `main.cpp`, where the OpenGL environment, GLUT window, audio system, and callback functions are initialized. Once initialization is complete, GLUT takes control through its event loop and continuously invokes registered callback functions.

The overall execution flow is shown below:

```
main()
|
+--> OpenGL Initialization
|
+--> Audio Initialization
```



```
|
+--> Game Initialization
|
+--> Register GLUT Callbacks
|
+--> Enter GLUT Main Loop
|
+--> Display Callback
|
+--> Timer Callback
|
+--> Keyboard Callback
|
+--> Special Key Callback
|
+--> Reshape Callback
```

The system continuously updates and renders the game at approximately 60 frames per second using a timer-based animation loop.

2.3 Game State Management

The game operates using a finite state machine (FSM) implemented through the `GameState` enumeration.

```
enum GameState
{
    HOME,
    PLAYING,
    GAME_OVER
};
```

The state transitions occur as follows:

```
HOME
|
| Press S
v
PLAYING
|
| Lives = 0
v
GAME_OVER
|
| Press R
v
HOME
```

This approach simplifies gameplay management by ensuring that only the logic associated with the active state is executed.

2.4 OpenGL Rendering Pipeline

The rendering process of Egg Drop Saga follows a structured pipeline that is executed repeatedly during gameplay.

Stage 1: Initialization

During program startup, the following operations are performed:

- GLUT initialization using `glutInit()`.
- Creation of the game window.
- Activation of RGB color mode.
- Activation of double buffering.
- Configuration of alpha blending.
- Setup of the orthographic projection.
- Initialization of game resources.
- Loading of audio files.

Stage 2: Input Processing

Player input is captured through GLUT callback functions.

- `keyboard()` handles standard keyboard input.
- `specialKeys()` handles arrow key input.

The input is forwarded to:

```
game.handleInput()  
game.handleSpecialInput()
```

These functions update the bucket position, game state, pause state, and audio settings.

Stage 3: Update Cycle

The timer callback executes every 16 milliseconds.

```
glutTimerFunc(16, timer, 0);
```

This produces approximately:

$$FPS = \frac{1000}{16} \approx 60$$

During each update cycle:

- Background animation is updated.
- Home screen animation is updated.
- Eggs are spawned if necessary.
- Egg positions are updated.
- Collision detection is performed.
- Score and life counters are updated.

Stage 4: Rendering

The display callback invokes:

```
game.render();
```

The rendering order is:

1. Clear frame buffer.
2. Draw background.
3. Draw chickens.
4. Draw eggs.
5. Draw bucket.
6. Draw score text.
7. Draw life indicators.
8. Draw state-specific screens.

Stage 5: Buffer Swapping

After rendering completes:

```
glutSwapBuffers();
```

The back buffer becomes visible while the next frame is rendered in the hidden buffer.

This double-buffering technique eliminates screen flickering and ensures smooth animation.

2.5 Coordinate System Design

Egg Drop Saga uses a two-dimensional orthographic coordinate system configured through:

```
gluOrtho2D(0, GAME_WIDTH, 0, GAME_HEIGHT);
```

The coordinate system is defined as follows:

- Origin (0,0) is located at the bottom-left corner.
- Positive X values extend toward the right.
- Positive Y values extend upward.
- All game objects are positioned using world coordinates.

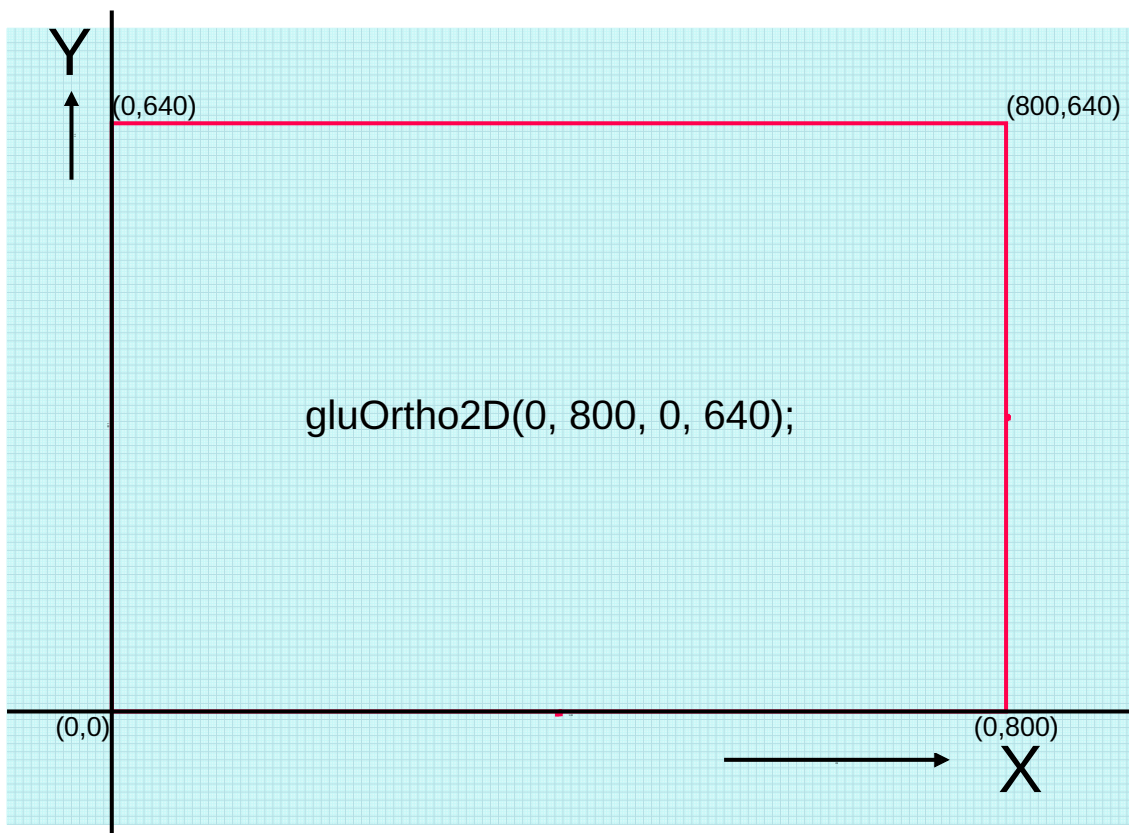


Figure 2.2: orthographics Graph of the Game Screen

Examples of object positioning:

- Bucket is initialized near the bottom of the screen.
- Chickens are positioned along the upper wire.
- Eggs spawn below randomly selected chickens.
- UI elements are positioned using fixed coordinates.

Chapter 3 Implementation

Chapter 4 Results and Output

Chapter 5 Discussion

Chapter 6 Conclusion

Chapter 7 Future Work

Chapter 8 References

Appendix